

Project Report: GetMeTheJob

1. Introduction & Idea Description

GetMeTheJob is a comprehensive, AI-driven career operating system. Unlike traditional resume builders or basic grammar checkers, GetMeTheJob acts as a persistent platform tailored for students and early-career professionals. The core idea is to move beyond the "one-shot" resume upload by providing continuous, personalized career guidance.

The system leverages large language models (LLMs) to automatically parse raw resume files, evaluate the candidate's existing skill set against industry-standard taxonomies for over 15 distinct job roles (such as Software Engineer, Product Manager, and Data Scientist), and generate a highly personalized, actionable roadmap to bridge any identified skill gaps. Finally, it provides an intelligent resume optimizer that rewrites weak bullet points to match specific job descriptions and exports a clean, recruiter-ready PDF.

2. Necessity & Problem Statement

The Problem

The modern job market, especially in tech, is highly competitive and relies heavily on Applicant Tracking Systems (ATS). Early-career professionals face three major hurdles:

1. **The Skills Blindspot:** Candidates often don't know exactly what technical or soft skills they are missing for their target roles.
2. **Analysis Paralysis:** Even when candidates know their gaps, the sheer volume of online courses and tutorials makes it difficult to prioritize *what* to learn next.
3. **Poor Resume Formatting:** Candidates struggle to translate their actual work into "impact statements" (Action Verb + Metric + Outcome) that pass ATS filters and impress human recruiters.

The Solution (Necessity)

There is a pressing need for a unified platform that acts as an automated career coach. GetMeTheJob solves this by automating the feedback loop: **Assess** → **Plan** → **Optimize**. By utilizing AI to structure unstructured resume data, we can provide deterministic skill-gap analysis, generate actionable learning roadmaps, and instantly optimize resume syntax—saving users hundreds of hours of manual research and formatting.

3. Working of the Project

GetMeTheJob is built on a modern, serverless web architecture using Next.js, PostgreSQL, and the Groq API for rapid LLM inference. The workflow consists of four primary modules:

A. Smart Profile Builder (Data Extraction)

- **Flow:** The user uploads a PDF or DOCX file. The system uses `unpdf` or `mammoth` to extract the raw text natively in the Node.js server environment.
- **AI Integration:** The raw text is passed to the Groq API (`gpt-oss-120b`). The AI is prompted via strict JSON formatting instructions to extract and map the text into structured arrays: skills, experience, projects, education, certifications, and achievements.
- **Storage:** This structured data is saved to a PostgreSQL database via Prisma ORM under the user's authenticated profile.

B. Readiness Engine

- **Flow:** The user selects a target role from a predefined taxonomy (e.g., "Data Scientist").
- **AI Integration:** The system compares the user's structured JSON profile against the JSON taxonomy for that role. The AI calculates an overall "ATS / Readiness Score" and explicitly categorizes skills into "Matched" and "Missing," identifying critical gaps holding the candidate back.

C. Action Roadmap Generator

- **Flow:** Based on the missing skills identified by the Readiness Engine, the AI generates a prioritized list of actionable steps (e.g., "Complete a React state management tutorial," "Build a REST API in Node.js").
- **UI Interaction:** These steps are rendered as a Kanban board (Pending, In Progress, Done). Users can move cards as they complete tutorials, making the application a persistent tracking tool rather than a static webpage.

D. Resume Optimizer & PDF Export

- **Flow:** The user pastes a specific target Job Description (JD).
 - **AI Integration:** The AI evaluates the user's existing profile against the JD, highlighting missing keywords. It then generates side-by-side (Before & After) suggestions, rewriting weak bullet points into strong, metrics-driven achievements.
 - **Live Editor:** The user can view these suggestions in a clean Diff view, apply them, and open a live WYSIWYG React-PDF editor. They can tweak formatting, upload a profile picture, and instantly download a beautiful, structured PDF.
-

4. Tech Stack

- **Frontend:** Next.js 16 (App Router), React 19, Tailwind CSS v4, shadcn/ui, Lucide Icons.
- **Typography:** Outfit (Next/Font) for modern, geometric readability.
- **Backend:** Next.js Server Actions & API Routes (Serverless architecture).

- **Database:** PostgreSQL managed via Prisma ORM.
 - **Authentication:** NextAuth (Auth.js) with secure session handling.
 - **AI Provider:** Groq API (Inference engine utilizing `openai/gpt-oss-120b` for near-instant responses).
 - **Document Processing:** `@react-pdf/renderer` (PDF generation), `unpdf` & `mammoth` (Document parsing).
-

5. Challenges Faced

1. Deterministic AI Outputs:

- *Challenge:* Standard LLMs tend to return conversational text (e.g., "Here is your parsed resume..."), which breaks frontend JSON parsers.
- *Solution:* We implemented strict prompt engineering and utilized `response_format: { type: "json_object" }` alongside Zod validation to ensure the AI only returned highly structured, database-ready JSON.

2. File Parsing in Next.js Server Environment:

- *Challenge:* Extracting text from PDFs in a Next.js 15+ Serverless/Turbopack environment is notoriously difficult. We initially attempted to use `pdfjs-dist`, but its modern versions crash in Node environments due to missing Browser APIs (DOMMatrix). We then migrated to the legacy `pdf-parse` library, but its internal Node.js file-system (`fs`) dependencies and CommonJS export structures caused fatal bundler errors (`TypeError: a is not a function`) when compiled for Vercel/Railway production environments.
- *Solution:* We completely replaced legacy parsers with **unpdf**, a modern, pure-JavaScript PDF extractor designed specifically for serverless and edge environments. Because `unpdf` holds the entire document in the server's RAM during processing, we implemented explicit memory management by manually loading the document via `getDocumentProxy`, extracting the text array, and immediately invoking `pdf.destroy()` to prevent production Out-Of-Memory (OOM) leaks.

3. React-PDF Layout Complexities:

- *Challenge:* Generating PDFs dynamically using `@react-pdf/renderer` required fighting against its internal Yoga flexbox engine, especially when trying to format dynamic multi-line contact details and user-uploaded base64 images without causing page-break overflows.
- *Solution:* We decoupled the heavy React state from the PDF viewer. By introducing a manual "Refresh Preview" button in the Live Editor, we prevented excessive re-renders that were causing the UI to freeze, and heavily utilized explicit `View` block wrappers to force the PDF engine to stack elements correctly.

4. **Taxonomy Scaling:**

- *Challenge:* Hardcoding skill requirements for every job role is unscalable and limits the platform's usefulness.
- *Solution:* We abstracted the role definitions into a dedicated `taxonomies.json` file. This allowed us to instantly scale from 1 role (Software Engineer) to 17 diverse roles (including QA Engineer, Product Manager, Growth Marketer, etc.) without altering the underlying database schema or AI logic.